Elective Course
Course Code: CS4103
Autumn 2026-27

Lecture #12

Artificial Intelligence for Data Science

Module-2: PROBLEM SOLVING BY SEARCH

Introduction to Informed Search [Part-IV] Local Search: Hill Climbing, Simulated Annealing
Introduction to Adversarial Search [Part-I]

Course Instructor:

Dr. Monidipa Das
Assistant Professor
Department of Computational and Data Sciences
Indian Institute of Science Education and Research Kolkata, India 741246

Hill-Climbing Search

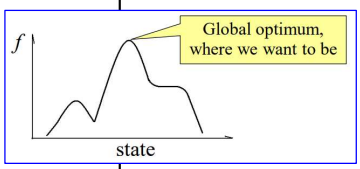
- "Like climbing Everest in thick fog with amnesia"

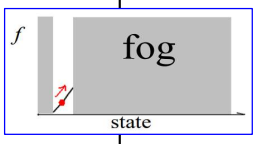
```
function HILL-CLIMBING( problem) returns a state that is a local maximum
   inputs: problem, a problem
   local vars: current, a node
               neighbor, a node

   current ← MAKE-NODE(INITIAL-STATE[problem])
   loop do
      neighbor ← a highest-valued successor of current
      if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
      current ← neighbor
   end
```

Best Successor

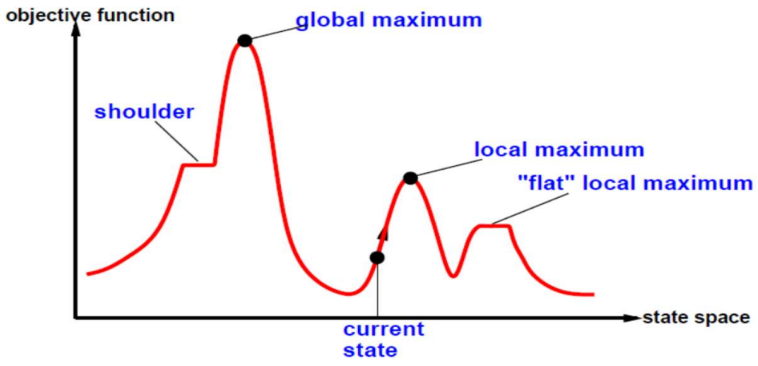
worse than





Hill Climbing in 1-D State-space Landscape

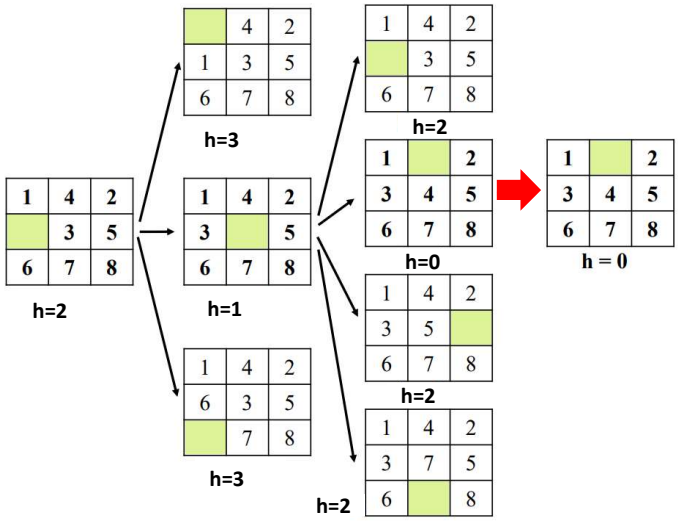
- Following is a one-dimensional state-space landscape in which elevation corresponds to the objective function.



- The aim is to find the global maximum. Hill-climbing search modifies the current state to try to improve it, as shown by the arrow

Hill Climbing Example

- **8-puzzle**



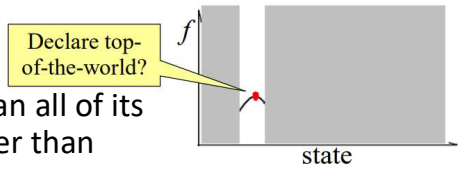


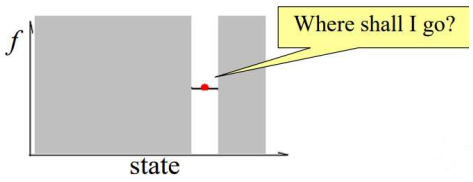
Heuristic function is Manhattan Distance

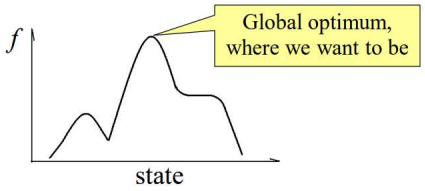
Hill Climbing: Disadvantages

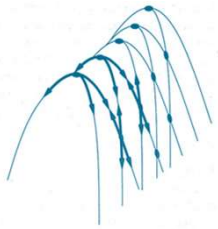


- **Local Maximum**
  - A state that is better than all of its neighbors, but not better than some other states far away
- **Plateau**
  - A flat area of the search space in which all neighboring states have the same value
- **Ridge**
  - Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate.





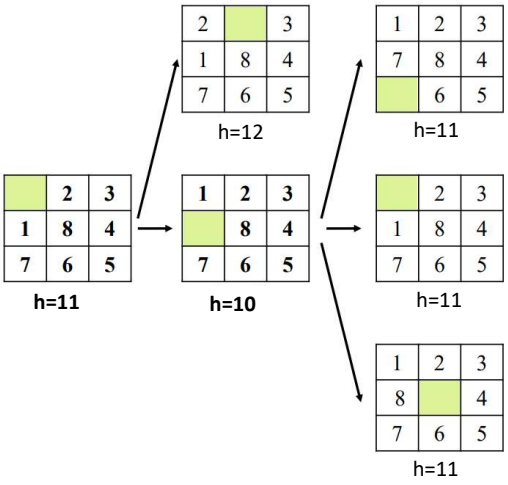




Hill Climbing Disadvantage: Example



- **8-puzzle: stuck at local optimum**



Heuristic function is Manhattan Distance

We are stuck with a local optimum

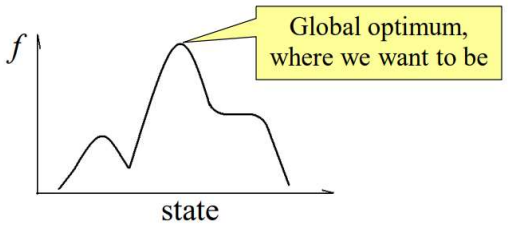


Goal State

Hill Climbing: Disadvantages



- **Ways Out**
  - Random selection of neighbor
  - Make a big jump to try to get in a new section.
  - Move in several directions at once



First-Choice Hill Climbing



- **First-choice hill climbing** implements stochastic hill climbing by generating successors randomly until one is generated that is better than the current state.
- This is a good strategy **when a state has many successors**.
- First-choice hill climbing is also **NOT complete**

Stochastic Hill Climbing



- **Stochastic hill climbing** chooses at random from among the uphill moves;
- The probability of selection can vary with the steepness of the uphill move.
- Stochastic hill climbing usually **converges more slowly** than steepest ascent, **but in some state landscapes, it finds better solutions**.
- Stochastic hill climbing is **NOT complete**, but it may be **less likely to get stuck.**

Random-Restart Hill Climbing



- **Random-Restart Hill Climbing** conducts a series of hill-climbing searches from randomly generated initial states, **until a goal is found.**
- Random-Restart Hill Climbing is complete if infinite (or sufficiently many) tries are allowed.
- If each hill-climbing search has a probability p of success, then the expected number of restarts required is 1/p.



Simulated Annealing

Another Local Search Technique

Annealing Process



- Annealing is a thermal process for obtaining low energy states of a solid in a heat bath.
- **Annealing Process**
  - Raising the temperature up to a very high level (melting temperature, for example), the atoms have a higher energy state and a high possibility to re-arrange the crystalline structure.
  - Cooling down slowly, the atoms have a lower and lower energy state and a smaller and smaller possibility to re-arrange the crystalline structure.

Analogy



- **Analogy –**
  - Metal ←→ Problem
  - Energy State ←→ Cost Function
  - Temperature ←→ Control Parameter
  - A completely ordered crystalline structure ←→ the optimal solution for the problem

Global optimal solution can be achieved as long as the cooling process is slow enough.

Simulated Annealing



- **Based on a metallurgical metaphor**
  - Start with a temperature set very high and slowly reduce it.
  - Run hill climbing with the twist that you can occasionally replace the current state with a worse state based on the current temperature and how much worse the new state is.
- **More formally…**
  - Generate a random new neighbor from current state.
  - If it is better take it.
  - If it is worse, then take it with some probability proportional to the temperature and the delta between the new and old states.

Simulated Annealing Algorithm




```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
          schedule, a mapping from time to "temperature"

  current ← MAKE-NODE(problem.INITIAL-STATE)
  for t = 1 to ∞ do
      T ← schedule(t)
      if T = 0 then return current
      next ← a randomly selected successor of current
      ΔE ← next.VALUE – current.VALUE
      if ΔE > 0 then current ← next
      else current ← next only with probability e^(ΔE/T)
```


Simulated Annealing Search



- Probability of a move decreases with the amount ΔE by which the evaluation is worsened
- A second parameter T is also used to determine the probability: high T allows more worse moves, T close to zero results in few or no bad moves
- *Schedule* input determines the value of T as a function of the completed cycles

Simulated Annealing Search



- **Idea:** improve hill-climbing by allowing occasional down-hill steps, to minimize the probability of getting stuck in a local maximum
- Down-hill steps taken randomly but with probability that decreases with time
- Probability controlled by a given annealing schedule for a temperature parameter T
- If schedule lowers T slowly enough, search is guaranteed to end in a global maximum
- **Catch:** it may take several tries with test problems to devise a good annealing schedule
- Simulated Annealing vs. Hill Climbing



Adversarial Search--- Games

Search vs. Games



- **Search- No adversary**
  - Solution is a path from start to goal, or a series of actions from start to goal
  - Heuristics and search techniques can find optimal solution
  - Evaluation function: estimate of cost from start to goal through given node
  - Actions have cost
  - Example: Path planning, Scheduling activities
- **Games- Adversary**
  - Solution is strategy
    - Strategy specifies move for every possible opponent reply
  - Time limits force an approximate solution
  - Evaluation function: evaluate "goodness" of game position
  - Board configurations have utility
  - Examples: Chess, Checkers, Othello, Backgammon

Why Study Games in AI?



- Problems are formalized
- Real world knowledge (common sense knowledge) is not too important
- Rules are fixed
- Adversary modeling is of general importance
  - opponent introduces uncertainty
  - programs must deal with the contingency problem
- Complexity of games?
  - number of nodes in a search tree (e.g., 10^{40} legal positions in chess)
  - specification is simple (no missing information, well-defined problem)

Types of games



	Deterministic	Chance
Perfect Information	Chess, Checkers, Othello, Go	Backgammon, Monopoly
Imperfect Information		Bridge, Poker

Not considered: Physical games like tennis, croquet, ice hockey, etc.

We restrict our analysis to a very specific set of games:
2-player zero-sum discrete finite deterministic games of **perfect information**

2-player zero-sum discrete finite deterministic games of perfect information



- **What does it mean?**
  - **2-player:** Two players involved (multi-agent environment)
  - **Zero-sum:** In any outcome of any game, Player A's gains equal player B's losses.
  - **Discrete:** All game states and decisions are discrete values.
  - **Finite:** Only a finite number of states and decisions.
  - **Deterministic:** No chance (no die rolls).
  - **Perfect information:** Both players can see the state, and each decision is made sequentially (no simultaneous moves).

Games as Adversarial Search



- **States:**
  - board configurations
- **Initial state:**
  - the board position and which player will move
- **Successor function:**
  - returns list of (action, state) pairs, each indicating a legal move and the resulting state
- **Terminal test:**
  - determines when the game is over
- **Utility function:**
  - gives a numeric value in terminal states

  (e.g., -1, 0, +1 for loss, tie, win)



Questions?